

Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Sistemas Operacionais
Prof: Mark Alan Junho Song

1. Compare os métodos de alocação de arquivos em disco por alocação contígua, encadeada e indexada, discutindo os seguintes aspectos:
 - a) facilidade de inserção e remoção de blocos
 - 1. Alocação Contígua**
 - **Como funciona:** O arquivo ocupa blocos sequenciais colados uns nos outros.
 - **Inserção/Remoção: MUITO RUIM E PESADA.**
 - **O motivo:** Como exige um espaço inteiro e contínuo, o arquivo não pode crescer dinamicamente. Para inserir algo novo (se não houver espaço vizinho vazio), o sistema precisa mover o arquivo inteiro para um novo "buraco" no disco.
 - 2. Alocação Encadeada**
 - **Como funciona:** O arquivo vira uma lista, onde cada bloco fica espalhado no disco e guarda um ponteiro indicando onde está o próximo.
 - **Inserção/Remoção: MUITO FÁCIL.**
 - **O motivo:** Basta manipular os ponteiros. Para remover, você apaga a ligação; para inserir, basta usar qualquer bloco vazio espalhado no disco e fazer o ponteiro do último bloco apontar para este novo.
 - 3. Alocação Indexada**
 - **Como funciona:** Um único bloco central (a tabela de índices) guarda todos os ponteiros de onde estão os blocos de dados daquele arquivo.
 - **Inserção/Remoção: MUITO FÁCIL.**
 - **O motivo:** Toda a modificação de espaço é resolvida em um só lugar. Para adicionar ou remover blocos do arquivo, basta inserir ou apagar o endereço correspondente dentro dessa tabela de alocação.
 - b) rapidez no acesso a registros
 - 1 - Alocação Contígua: Extremamente Rápida.** Como os blocos estão gravados fisicamente colados uns nos outros no disco, a cabeça de leitura faz o menor movimento possível (tempo de busca quase zero) para ler os dados sequencialmente.
 - 2 - Alocação Encadeada: Muito Lenta.** Para ler um registro específico, o sistema precisa ler o bloco atual na mídia física, achar o ponteiro, ler o próximo bloco, e repetir isso bloco por bloco até achar o destino.
 - 3 - Alocação Indexada: Rápida.** O sistema lê o bloco de índice e descobre de imediato o endereço exato do bloco de dados que precisa, acessando-o diretamente sem ler os intermediários.
 - c) formas de acesso possíveis (seqüencial/randômica)
 - 1 - Alocação Contígua: Suporta Ambos (Sequencial e Randômico/Direto).** Como os blocos são contínuos, o sistema calcula a posição de qualquer registro instantaneamente e acessa o bloco diretamente, sem precisar passar pelos blocos anteriores.
 - 2 - Alocação Encadeada: Apenas Sequencial.** Como o ponteiro para o bloco seguinte está guardado fisicamente dentro do bloco anterior, é impossível saltar direto para o final do arquivo sem ler toda a cadeia de ponteiros desde o início.
 - 3 - Alocação Indexada: Suporta Ambos (Sequencial e Randômico/Direto).** A tabela de índices centralizada permite que o sistema operacional consulte o índice de um arquivo e pule diretamente para o bloco desejado, eliminando dependências sequenciais.

Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Sistemas Operacionais
Prof: Mark Alan Junho Song

d) necessidade de armazenamento de informações adicionais para manutenção do arquivo

Contígua: Basta o **endereço do bloco inicial** e o **tamanho total** do arquivo (ou bloco final). É o modelo mais simples.

Encadeada: O cabeçalho guarda apenas o endereço do primeiro bloco. Porém, **dentro de cada bloco**, você deve reservar um espaço (ponteiro) para o endereço do próximo bloco.

Indexada: Você precisa de um **bloco de índice** (uma tabela) que liste todos os endereços de todos os blocos que compõem aquele arquivo. O cabeçalho aponta para esse bloco de índice.

e) fragmentação interna e externa

A fragmentação ocorre quando o espaço de armazenamento não é utilizado de forma eficiente.

Fragmentação Externa

Ocorre quando há espaço livre total no disco (ou na memória) suficiente para alocar um arquivo, mas esse espaço está **dividido em pequenos "buracos" não contíguos**, impedindo que o arquivo seja armazenado porque ele exige um espaço único e contínuo.

- **Onde ocorre:** Principalmente na alocação **contígua**.

Fragmentação Interna

Ocorre quando o sistema aloca um bloco de tamanho fixo para um arquivo, mas o arquivo não ocupa todo esse espaço. O espaço restante dentro do bloco torna-se "inutilizável" para outros fins.

- **Exemplo:** Se o sistema usa blocos de 4KB e você salva um arquivo de 1KB, os outros 3KB estão reservados e "perdidos" (fragmentação interna).
- **Onde ocorre:** Pode ocorrer em qualquer método que utilize blocos fixos, sendo uma característica comum de sistemas que visam otimizar a velocidade em detrimento do uso preciso do espaço.

2. Descreva o processo de transferência de dados de um disco para a memória, com acesso direto à memória (DMA). Deixe claros os papéis da CPU, driver, controladoras e barramentos neste processo.

O processo de Acesso Direto à Memória (DMA) alivia o gargalo computacional transferindo enormes massas de dados poupando a intervenção massiva de instruções. A operação inicia-se mediante requisição da **CPU**, que avisa o **driver** lógico do SO sobre o que deseja ler (endereços lógicos na estrutura). Este software (driver) atua comunicando-se e injetando parâmetros na **controladora do dispositivo (hardware)** de disco informando os blocos, e paralelamente seta a controladora especial de DMA detalhando onde na memória os dados deverão repousar. Pelos **barramentos de dados e controle**, a controladora do disco extrai os trechos magnéticos para o seu *buffer*, e o controlador de DMA intercede tomando a posse prioritária do barramento, copiando os bytes em trânsito direto para a memória principal. Durante este trânsito autônomo, a CPU opera em processos paralelos. O ciclo fecha quando a controladora alerta a CPU gerando uma interrupção.

Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Sistemas Operacionais
Prof: Mark Alan Junho Song

3. O que é uma FAT (Tabela de alocação de arquivos)? Qual seu papel no método de alocação encadeado?

A FAT (*File-Allocation Table*) representa um vetor avançado alocado de preferência integralmente no escopo da memória principal, popularizado estruturalmente pelos sistemas MS-DOS. A sua atuação consiste especificamente no aprimoramento puro do método de alocação encadeada. Em vez de o sistema esconder fragmentado o ponteiro do bloco próximo diretamente nos arquivos de dados no longo e devagar disco, a FAT concentra as ramificações de encadeamento separadamente nesse bloco unitário na RAM. Seu papel resolve a maior fraqueza do formato encadeado: remove o tempo imenso de leitura da mídia para consultas rápidas sequenciais propiciando um contorno eficiente de acesso randômico aos registros.

4. Um disco tem oito setores de 512 bytes por trilha, e uma taxa de rotação de 300rpm. Quanto tempo leva para a leitura de todos os setores da trilha em ordem assumindo que o braço já está corretamente posicionado, que meia (0.5) rotação é necessária para localizar o setor 0 (zero), e que a taxa de transferência do bloco demanda 15 msegundos? Seria possível adotar uma nova política para melhorar a performance? Faça o mesmo cálculo para a nova situação (interleaving simples). E se a taxa de transferência demandar 40 msegundos o que aconteceria? Seria possível adotar uma nova política para melhorar a performance? (interleaving duplo).

Tempo de rotação completa: $60s(60.000) / \text{RPM}$

Tempo de um setor passar sob a cabeça: $\text{Trotação} / \text{N}^\circ \text{ de setores por trilha}$

Tempo de latência rotacional (busca do setor inicial): $\text{Frações de rotação} \times \text{Trotação}$

Quantos setores o disco gira durante uma transferência: $T_{\text{transferencia}} / T_{\text{setor}}$

Tempo total SEM interleaving (leitura sequencial ok): $T_{\text{latencia}} + (\text{N}^\circ \text{ setores} \times T_{\text{setor}})$

Tempo total SEM interleaving (quando transferência > T_{setor}): $T_{\text{latencia}} + T_{\text{transferencia}} + (\text{N}^\circ \text{setores} - 1) \times (\text{Trotação} - T_{\text{transf}} + T_{\text{setor}})$

Tempo total COM interleaving simples: $T_{\text{latencia}} + \text{N}^\circ \text{setores} \times (T_{\text{setor}} \times 2)$

Tempo total COM interleaving duplo: $T_{\text{latencia}} + \text{N}^\circ \text{setores} \times (T_{\text{setor}} \times 3)$

Resposta: RPM = 300; 512 bytes por trilha sendo 8 trilhas; 0,5 rotação para o setor 0;

Taxa de transferência do bloco = 15 ms.

Tempo de rotação completa = $60.000/300 = 200\text{ms}$

Meia rotação = 100ms

Se é 15ms por setor para transferência e são 8 setores = 120

$200/8 = 25\text{ms}$ tempo que passa por setor é maior que o de transferência então não é necessário reordenar para melhorar o tempo

O tempo total para leitura seria = $100\text{ms} + (25 \times 8) = 300\text{ms}$,

Não, nesse caso a única forma de melhorar seria diminuir o RPM.

Como agora a taxa de transferência é 40ms o tempo de transferência é maior que o de rotação, então é necessário que ao acabar o setor 0 ele esteja no setor 1, para isso um interleaving simples resolve pois ele vai acabar o 1 em 40 ms e vai rodar 50 ms então podemos fazer assim a ordem:

0 4 1 5 2 6 3 7 4, e o novo tempo seria: $T_{\text{latencia}} + \text{N}^\circ \text{setores} \times (T_{\text{setor}} \times 2) =$

$100\text{ms} + (8 \times (25 \times 2)) = 500\text{ms}$

Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Sistemas Operacionais
Prof: Mark Alan Junho Song

Com interleaving duplo ficaria assim a ordem: 0,3,6,1,4,7,2,5, e o tempo seria de Tlatência + N°setores
 $X (T_{setor} \times 3) = 100 + (8 \times (25 \times 3)) = 700ms$

5. Foi sugerido que a primeira parte de cada arquivo UNIX fosse mantido no mesmo bloco de disco com o seu i-node. Que benefícios esta política gera?

O sistema UNIX utiliza um esquema combinado muito famoso baseado em *i-nodes* (nós de índice). Um i-node é uma pequena estrutura que guarda os metadados do arquivo (proprietário, permissões, datas) e um conjunto de ponteiros diretos e indiretos para os blocos de dados.

- O exercício sugere guardar a primeira parte dos dados do arquivo *dentro* do próprio bloco do i-node. O grande benefício dessa política é o aumento drástico de desempenho para arquivos muito pequenos (que são a maioria no sistema). Se o arquivo couber inteiro dentro do i-node, o sistema não precisará fazer uma segunda busca no disco para ler os dados, economizando tempo de leitura e reduzindo a fragmentação.

R: Manter a primeira parte do arquivo no mesmo bloco do i-node reduz de 2 para 1 o número de acessos ao disco para arquivos pequenos, pois i-node e dados são lidos em uma única operação. Como a maioria dos arquivos UNIX é pequena, isso gera um ganho expressivo de desempenho geral. Para arquivos maiores, apenas a leitura inicial é acelerada, e o restante continua usando os ponteiros indiretos normalmente.

1- Responsabilidade do SO com arquivos

O que o SO faz

Mantém uma estrutura de diretórios com informação sobre todos os arquivos. Essa estrutura reside em disco.

Operações de arquivo que o SO gerencia: criar, gravar, ler, reposicionar (seek), apagar, truncar, abrir e fechar.

Abrir (Fi): pesquisa o diretório, move a entrada para memória. Fechar (Fi): move a entrada de volta para o diretório em disco.

Controla proteção: quem pode ler, escrever ou executar cada arquivo (bits RWX por proprietário, grupo e público).

Gerencia alocação de espaço em disco (contígua, encadeada ou indexada).

Resumo: o SO é responsável por criar a ilusão de arquivos sobre blocos de disco — gerenciando espaço, nomes, proteção e acesso.

2- Estruturas de arquivos

Tipos de estrutura (do cap. 11)

Nenhuma: arquivo é simplesmente uma sequência de bytes. O programa interpreta o conteúdo.

Estrutura de registro simples: linhas de tamanho fixo ou variável.

Estrutura complexa: documentos formatados (PDF, DOCX) ou arquivo de carga realocável. Quem define pode ser o SO ou o programa.

Tipos de conteúdo

Dados numéricos, dados de caractere, dados binários e programas.

Ponto-chave: o arquivo é um "espaço de endereçamento lógico contíguo" — mas quem define o formato interno é o programa ou o SO.

Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Sistemas Operacionais
Prof: Mark Alan Junho Song

3- Indexação de arquivo

Alocação indexada

Bloco de índice: reúne todos os ponteiros para os blocos de dados em um único bloco especial.

O diretório guarda apenas o número do bloco de índice. Para acessar o i -ésimo bloco, lê-se o índice na posição i .

Permite acesso randômico e alocação dinâmica sem fragmentação externa.

Custo: mesmo um arquivo de 2 blocos precisa de um terceiro bloco extra apenas para os ponteiros.

Esquema combinado (UNIX)

Blocos diretos: apontam direto para dados (arquivos pequenos).

Indireto simples: bloco de índice aponta para blocos de dados.

Indireto duplo: bloco aponta para blocos de índice, que apontam para dados.

Indireto triplo: três níveis de indireção — para arquivos muito grandes.

O esquema UNIX com 4K bytes/bloco usa blocos diretos + 3 níveis de indireção para suportar arquivos de qualquer tamanho.

4- Tamanho de arquivo

Atributos relacionados ao tamanho

Tamanho é um atributo do arquivo mantido na estrutura de diretório (junto com nome, tipo, localização, proteção, data).

Na alocação contígua: o SO precisa saber apenas o bloco inicial e o número de blocos (= tamanho).

Na alocação encadeada: o tamanho pode crescer dinamicamente — novos blocos são alocados conforme necessidade.

Na alocação contígua: arquivos não podem crescer após criados (limitação importante).

Fragmentação relacionada

Fragmentação interna: bloco parcialmente usado no final do arquivo — espaço alocado > espaço necessário.

5- Controladora de I/O

Contexto no cap. 9 e 11

O armazenamento auxiliar (disco) precisa prover acesso direto às imagens de memória dos processos (usado no swapping).

O sistema de arquivos reside em memória secundária (disco). A estrutura de diretório e os arquivos ficam em disco.

Swapping: a maior parte do tempo de swap é de transferência. O tempo total é proporcional à quantidade de memória transferida entre RAM e disco.

O File Control Block (FCB) é a estrutura de armazenamento que contém as informações sobre o arquivo — comunicação entre SO e hardware de I/O.

Atenção: os PDFs têm pouco detalhe sobre controladora de I/O em si. Essa questão pode se referir ao papel da controladora no acesso ao disco durante operações de arquivo.

6- Metadados de arquivos

Atributos (metadados) mantidos pelo SO

Nome: única informação mantida na forma legível ao usuário.

Tipo: necessário em sistemas que suportam vários tipos de arquivo.

Localização: ponteiro para a posição do arquivo no dispositivo.

Tamanho: tamanho corrente do arquivo.

Proteção: controla quem pode ler, escrever ou executar.

Data e identificação do usuário: dados para proteção, segurança e monitoração de uso.

Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Sistemas Operacionais
Prof: Mark Alan Junho Song

Todas essas informações ficam na estrutura de diretórios, residente em disco. No UNIX, ficam no inode (FCB).

Journaling (metadados de sistema)

Journaling: serviço de log que registra mudanças antes de gravá-las. Guarda metadados e dados que serão escritos. Usado no Ext3, Ext4.

7- Como o SO vê os arquivos

Visão lógica vs. física

Para o SO, um arquivo é um espaço de endereçamento lógico contíguo — independente de como os blocos estão fisicamente no disco.

O SO vê o arquivo através de sua entrada no diretório: nome, ponteiro para os blocos, tamanho, proteção. A estrutura interna do arquivo (se é texto, binário, registro) não é necessariamente conhecida pelo SO — pode ser o programa que interpreta.

Organização de diretórios

Um nível: único diretório para todos. Problema: conflito de nomes.

Dois níveis: diretório separado por usuário. Permite mesmo nome para usuários distintos.

Árvore: hierarquia com caminhos absolutos e relativos. Estrutura usada hoje (Linux, Windows).

Grafo: permite arquivos compartilhados (links). Usa contadores de referência para evitar ponteiros pendentes.

8- Conta seek — cálculo

O que é seek

Seek (reposicionar ponteiro): operação que move o ponteiro de leitura/escrita para uma posição específica no arquivo, sem ler ou escrever dados.

Cálculo típico em alocação indexada

Para acessar o byte na posição P em um arquivo com blocos de tamanho B:

Número do bloco = $P \div B$ Deslocamento dentro do bloco = $P \bmod B$

Em indireto simples: 1 acesso ao bloco de índice + 1 acesso ao bloco de dados = 2 acessos.

Em indireto duplo: 3 acessos. Em indireto triplo: 4 acessos. Blocos diretos: 1 acesso.

Analogia com paginação (cap. 9)

Na tabela de páginas simples: 2 acessos à memória (1 para tabela + 1 para dado). A TLB reduz para ~1 acesso efetivo.

Para a prova: identifique o nível de indireção necessário e some os acessos a disco.

9- Tamanho máximo de arquivo — cálculo

Esquema UNIX (4K bytes/bloco, ponteiro = 4 bytes)

Cada bloco tem 4096 bytes. Um bloco de índice cabe $4096 \div 4 = 1024$ ponteiros.

Contribuição de cada nível

Blocos diretos (ex: 12): $12 \times 4K = 48$ KB Indireto simples: $1024 \times 4K = 4$ MB Indireto duplo: $1024 \times 1024 \times 4K = 4$ GB Indireto triplo: $1024 \times 1024 \times 1024 \times 4K = 4$ TB

Tamanho máximo total $\approx 48KB + 4MB + 4GB + 4TB \approx 4$ TB (dominado pelo nível triplo).

Outros esquemas

Ext2: suportava até 2 TB de disco (mencionado no PDF).

FAT32: tamanho máximo de arquivo depende do número de bits da entrada FAT.

Para a prova: calcule quantos ponteiros cabem em um bloco (tamanho do bloco $\div$ tamanho do ponteiro) e multiplique pelos níveis de indireção.

Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Sistemas Operacionais
Prof: Mark Alan Junho Song

1

Compare alocação contígua, encadeada e indexada: inserção/remoção, velocidade, acesso, armazenamento extra, fragmentação.

Contígua: acesso randômico rápido, sem ponteiros extras, mas não cresce e causa fragmentação externa. Encadeada: cresce facilmente, sem fragmentação externa, mas só acesso sequencial e gasta espaço com ponteiros em cada bloco.

Indexada: acesso randômico + crescimento dinâmico, mas gasta um bloco extra inteiro para os ponteiros (índice). Não há fragmentação externa.

2

Descreva a transferência de dados do disco para a memória via DMA. Papéis de CPU, driver, controladoras e barramentos.

DMA (Direct Memory Access): a CPU programa a controladora DMA com endereço de memória destino, endereço do disco e quantidade de bytes. A controladora DMA assume o barramento e transfere os dados direto para a memória RAM sem intervenção da CPU. Ao terminar, envia uma interrupção para a CPU. O driver intermediou o pedido inicial; a controladora de disco leu os setores; a controladora DMA moveu os dados; o barramento conectou tudo.

3

O que é FAT? Qual seu papel na alocação encadeada?

FAT (File Allocation Table): é uma tabela mantida pelo SO que fica no início do disco. Cada entrada corresponde a um bloco do disco e contém o número do próximo bloco do arquivo (ou um marcador de fim). Resolve o problema do acesso randômico na alocação encadeada: em vez de percorrer os blocos um a um, o SO lê a FAT inteira na memória e navega pela cadeia de ponteiros sem acessar o disco. Usado no MS-DOS, FAT16, FAT32.

4

Cálculo de tempo de leitura de trilha com interleaving (simples e duplo). Disco: 8 setores $\times$ 512 bytes, 300 rpm, 0,5 rotação para localizar setor 0.

Dados: 300 rpm $\rightarrow$ 1 rotação = 200 ms. 8 setores $\rightarrow$ 1 setor = 25 ms. Localizar setor 0 = 0,5 rot = 100 ms.

Sem interleaving: lê setor 0 (15 ms transferência) $\rightarrow$ disco girou 15 ms $\rightarrow$ já passou para o setor seguinte $\rightarrow$ precisa esperar quase 1 rotação. Total $\approx$ muito alto.

Interleaving simples: setores intercalados de 1 em 1 (0, 4, 1, 5, 2, 6, 3, 7). Após transferir um setor, o próximo a ser lido é o que está chegando na cabeça. Reduz drasticamente o tempo total.

Interleaving duplo (40 ms): intercala de 2 em 2, pois a transferência é mais lenta. O padrão de ordem de setores físicos é ajustado para que o próximo setor lógico chegue sob a cabeça exatamente quando a transferência terminar.

5

Benefícios de manter a primeira parte de cada arquivo UNIX no mesmo bloco de disco que seu i-node.

I-node (inode): estrutura de metadados do UNIX que contém atributos do arquivo e ponteiros para os blocos de dados.

Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Sistemas Operacionais
Prof: Mark Alan Junho Song

Benefícios de co-localizar inode + início do arquivo:

- Menos acessos a disco: normalmente abrir um arquivo exige 1 acesso ao inode + 1 acesso ao primeiro bloco de dados. Juntos, vira 1 acesso.
- Arquivos pequenos ficam inteiros no bloco do inode, economizando blocos extras.
- Melhor desempenho de seek: a cabeça do disco já está posicionada onde estão os dados.

Aqui estão todas com as respostas corretas:

Q1 — Responsabilidade do SO Quando um processo executa abrir(arquivo.txt) e depois fechar(arquivo.txt), o que o SO faz em cada uma dessas operações? Onde as informações do arquivo ficam em cada momento?

Resposta: No abrir, o SO pesquisa a estrutura de diretório no disco pela entrada do arquivo e move seus **metadados** (nome, localização, tamanho, proteção) para a memória — não o conteúdo do arquivo. No fechar, o SO move essa entrada de metadados de volta para a estrutura de diretório em disco. O conteúdo do arquivo em si só é lido/escrito quando há operações explícitas de leitura e escrita, não no abrir/fechar.

Q2 — Estrutura de arquivos Um arquivo de texto e um arquivo executável (.exe) têm estruturas internas diferentes. Do ponto de vista do SO, qual é a diferença entre eles? Quem é responsável por interpretar essa estrutura?

Resposta: Do ponto de vista do SO, **não há diferença**: ambos são sequências de bytes. O SO não interpreta o conteúdo interno dos arquivos. Quem interpreta é o **programa**: o editor de texto trata os bytes como caracteres, o loader do SO trata o .exe como código executável. O SO usa o atributo **tipo** apenas como informação, sem analisar o conteúdo.

Q3 — Indexação de arquivo Explique como funciona a alocação indexada. Por que ela resolve um problema que a alocação encadeada tem, e qual é o custo dessa solução?

Resposta: Na alocação indexada, todos os ponteiros para os blocos de dados do arquivo são reunidos em um único **bloco de índice**. O diretório guarda apenas o endereço desse bloco de índice. Para acessar qualquer bloco do arquivo, basta ir ao índice e pegar o ponteiro diretamente, sem percorrer uma cadeia. Isso resolve o problema da alocação encadeada, que só permite **acesso sequencial** (é preciso seguir os ponteiros bloco a bloco). O custo é que mesmo um arquivo pequeno de 2 blocos precisa de um **terceiro bloco extra** apenas para armazenar os ponteiros.

Q4 — Tamanho de arquivo Por que na alocação contígua um arquivo não pode crescer depois de criado? O que acontece se o usuário tentar adicionar dados a um arquivo que já ocupa todo o espaço que lhe foi alocado?

Resposta: Na alocação contígua, o arquivo ocupa um conjunto de blocos consecutivos no disco. Como o espaço vizinho geralmente está ocupado por outro arquivo, não é possível estender o arquivo sem realocar tudo para um espaço maior — o que seria muito custoso. Por isso, sistemas com alocação contígua simplesmente **não permitem crescimento** após a criação. Se o usuário tentar adicionar dados, o SO retorna um **erro de espaço insuficiente**.

Q5 — Controladora de I/O Descreva o que acontece desde o momento em que o SO pede a leitura de um bloco do disco até os dados chegarem na memória RAM, usando DMA. Quem faz o quê nesse processo?

Resposta: A CPU programa a controladora DMA com três informações: endereço do bloco no disco, endereço de destino na memória RAM e quantidade de bytes. O **driver** intermediou esse pedido. A

Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Sistemas Operacionais
Prof: Mark Alan Junho Song

controladora de disco lê os dados magneticamente e os coloca em seu buffer. A **controladora DMA** então assume o controle do **barramento** e transfere os dados direto do buffer do disco para a memória RAM, sem intervenção da CPU. Quando termina, a controladora DMA envia uma **interrupção para a CPU**, avisando que os dados já estão disponíveis na memória.

Q6 — Metadados de arquivos Liste os metadados que o SO mantém sobre um arquivo e diga onde eles ficam armazenados. O nome do arquivo é diferente dos outros atributos — por quê?

Resposta: Os metadados são: **nome, tipo, localização, tamanho, proteção** (quem pode ler, escrever e executar) e **data/identificação do usuário**. Todos ficam armazenados na **estrutura de diretórios, residente em disco** (no UNIX, dentro do inode). O nome é diferente dos demais porque é a **única informação mantida em forma legível ao usuário** — todos os outros atributos são usados internamente pelo SO e não precisam ser legíveis.

Q7 — Como o SO vê os arquivos Um arquivo tem seus blocos espalhados em posições físicas não contíguas no disco. Como o SO apresenta esse arquivo para o programa que o lê? O programa precisa saber dessa fragmentação?

Resposta: O SO apresenta o arquivo como um **espaço de endereçamento lógico contíguo** — uma sequência ordenada de bytes, independente de como os blocos estão fisicamente no disco. O SO usa suas estruturas internas (bloco de índice, tabela de diretório, inode) para montar essa sequência lógica a partir dos blocos físicos espalhados. O programa **não precisa saber da fragmentação**: para ele, o arquivo é um fluxo contínuo de dados.

Q8 — Conta seek Um arquivo usa alocação indexada com blocos de 512 bytes e ponteiros de 4 bytes. Quantos acessos a disco são necessários para ler o byte na posição 300.000? Mostre o raciocínio passo a passo.

Resposta:

- Ponteiros por bloco de índice: $512 \div 4 = 128$ ponteiros
- Bloco lógico onde está o byte: $300.000 \div 512 = \text{bloco } 585$
- Indireto simples suporta: $128 \text{ blocos} \rightarrow 585 > 128$, não está aqui
- Indireto duplo suporta: $128 \times 128 = 16.384 \text{ blocos} \rightarrow 585 < 16.384$ ✓
- O byte está no **indireto duplo**, que exige:
 - 1 acesso ao bloco de índice externo
 - 1 acesso ao bloco de índice interno
 - 1 acesso ao bloco de dados
- **Total: 3 acessos a disco**

Q9 — Tamanho máximo de arquivo Um sistema usa o esquema UNIX com blocos de 4 KB e ponteiros de 4 bytes. Há 12 blocos diretos, 1 indireto simples, 1 indireto duplo e 1 indireto triplo. Qual é o tamanho máximo de arquivo suportado? Calcule cada nível separadamente.

Resposta:

- Ponteiros por bloco: $4.096 \div 4 = 1.024$ ponteiros
- **12 blocos diretos:** $12 \times 4 \text{ KB} = 48 \text{ KB}$
- **Indireto simples:** $1.024 \times 4 \text{ KB} = 4 \text{ MB}$
- **Indireto duplo:** $1.024 \times 1.024 \times 4 \text{ KB} = 4 \text{ GB}$
- **Indireto triplo:** $1.024 \times 1.024 \times 1.024 \times 4 \text{ KB} = 4 \text{ TB}$
- **Total $\approx 4 \text{ TB}$** (dominado pelo nível triplo)